04 — Cross-Domain Composition

Architecture

A typical cross-domain system has components running at different speeds, connected through buffering:

Domain	Components
Fast	ADC, SampleBuffer
Slow	Processor, ResultStore
Bridge	FIFO (shared labels)

Data flows: ADC $\xrightarrow{\texttt{sample}}$ Buffer $\xrightarrow{\texttt{fifo\_push}}$ FIFO $\xrightarrow{\texttt{fifo\_pop}}$ Processor $\xrightarrow{\texttt{store\_result}}$ Store.

Shared Labels as Sync Points

Cross-domain boundaries are enforced by shared labels that act as handshake signals:

- `fifo_push` — Buffer ↔ FIFO
- `fifo_pop` — FIFO ↔ Processor
- `store_result` — Processor ↔ Store

Each pair synchronizes only on its shared labels; other components step independently.

Flat Async Workaround

Since hierarchical composition is not yet supported, model cross-domain systems as a single flat async composition:

```
asynchronous system {
    members [
        ADC,
        SampleBuffer,
        Processor,
        ResultStore
    ];
}
```

All components are listed as direct members. Domain boundaries emerge from the shared-label topology, not from nesting.

Design Pattern: Pairwise Sync

Use **unique shared labels** between each pair of communicating components to create sync barriers:

- Name labels by the pair: `adc_buf_sample`, `buf_fifo_push`, etc.
- Avoid giving the same label to unrelated pairs — it forces unwanted synchronization.
- This creates a chain of local handshakes that preserves domain isolation.

Verify Components Individually

Evaluating properties over **individual automata** (or small sub-compositions) gives more precise results:

```
formula buf_safe {
    over SampleBuffer;
    body = nu X. safe && ([] X);
}
```

- Smaller state space ⇒ faster fixpoint computation.
- Easier to pinpoint which component violates a property.
- Compose and re-verify only when component-level properties pass.

Hierarchical Composition

Mununu does **not yet support** referencing a composition as a member of another composition:

```
// NOT supported:
asynchronous outer {
    members [fast_domain, slow_domain];
}
```

Workaround: flatten all components into a single `asynchronous` block and rely on shared-label topology to enforce domain boundaries.